Algorytm Listy Dwukierunkowej

Generated by Doxygen 1.18.0

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 DoublyLinkedList Class Reference . . . 5

3.1.1 Detailed Description . . . 6

3.1.2 Constructor & Destructor Documentation . . . 6

3.1.2.1 DoublyLinkedList() . . . 6

3.1.2.2 ~DoublyLinkedList() . . . 6

3.1.3 Member Function Documentation . . . 6

3.1.3.1 begin() . . . 6

3.1.3.2 clear_list() . . . 7

3.1.3.3 display_list() . . . 7

3.1.3.4 display_next_element() . . . 7

3.1.3.5 display_prev_element() . . . 7

3.1.3.6 display_reversed_list() . . . 7

3.1.3.7 end() . . . 7

3.1.3.8 pop_back() . . . 8

3.1.3.9 pop_front() . . . 8

3.1.3.10 pop_wherever() . . . 8

3.1.3.11 push_back() . . . 8

3.1.3.12 push_front() . . . 9

3.1.3.13 push_wherever() . . . 9

3.2 Factory Class Reference . . . 9

3.2.1 Detailed Description . . . 10

3.2.2 Member Function Documentation . . . 10

3.2.2.1 createNode() . . . 10

3.3 Iterator Class Reference . . . 10

3.3.1 Detailed Description . . . 11

3.3.2 Constructor & Destructor Documentation . . . 11

3.3.2.1 Iterator() . . . 11

3.3.3 Member Function Documentation . . . 11

3.3.3.1 operator"!=() . . . 11

3.3.3.2 operator*() . . . 12

3.3.3.3 operator++() . . . 12

3.3.3.4 operator--() . . . 12

3.4 Node Class Reference . . . 12

3.4.1 Detailed Description . . . 13

3.4.2 Constructor & Destructor Documentation . . . 13

3.4.2.1 Node() . . . 13

4 File Documentation **15**

4.1 DoublyLinkedList.cpp File Reference . . . 15

4.1.1 Detailed Description . . . 15

4.2 DoublyLinkedList.h . . . 15

4.3 Factory.cpp File Reference . . . 16

4.3.1 Detailed Description . . . 16

4.4 Factory.h . . . 16

4.5 Iterator.cpp File Reference . . . 16

4.5.1 Detailed Description . . . 16

4.6 Iterator.h . . . 17

4.7 main.cpp File Reference . . . 17

4.7.1 Detailed Description . . . 17

4.7.2 Function Documentation . . . 17

4.7.2.1 main() . . . 17

4.8 Node.cpp File Reference . . . 18

4.8.1 Detailed Description . . . 18

4.9 Node.h . . . 18

Index **19**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

DoublyLinkedList
: Główna klasa zarządzająca listą dwukierunkową 5

Factory
: Klasa implementująca wzorzec Factory 9

Iterator
: Klasa implementująca wzorzec Iterator dla listy dwukierunkowej 10

Node
: Klasa reprezentująca pojedynczy węzeł listy dwukierunkowej 12

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

DoublyLinkedList.cpp
    Implementacja metod klasy DoublyLinkedList 15
DoublyLinkedList.h 15
Factory.cpp
    Implementacja metod klasy Factory 16
Factory.h 16
Iterator.cpp
    Implementacja metod klasy Iterator 16
Iterator.h 17
main.cpp
    Plik główny programu testujący implementację dwukierunkowej listy powiązanej 17
Node.cpp
    Plik zawierający implementację metod klasy Node 18
Node.h 18

Chapter 3

Class Documentation

3.1 DoublyLinkedList Class Reference

Główna klasa zarządzająca listą dwukierunkową.

`#include <DoublyLinkedList.h>`

Public Member Functions

- DoublyLinkedList ()
  Konstruktor klasy.
- ~DoublyLinkedList ()
  Destruktor klasy.
- **DoublyLinkedList** (const DoublyLinkedList &)=delete
  Blokowanie kopiowania listy, różne obiekty nie będą współdzielić tych samych węzłów.
- DoublyLinkedList & **operator=** (const DoublyLinkedList &)=delete
- void push_front (int value)
  Dodaje nowy element na sam początek listy.
- void push_back (int value)
  Dodaje nowy element na sam koniec listy.
- void push_wherever (int index, int value)
  Wstawia nowy element pod wskazany indeks.
- void pop_front ()
  Usuwa pierwszy element z listy i zwalnia jego pamięć.
- void pop_back ()
  Usuwa ostatni element z listy i zwalnia jego pamięć.
- void pop_wherever (int index)
  Usuwa element znajdujący się pod wskazanym indeksem.
- void display_list ()
  Wyświetla zawartość całej listy od początku do końca.
- void display_reversed_list ()
  Wyświetla zawartość całej listy w odwrotnej kolejności (od końca do początku).
- void display_next_element ()
  Wyświetla następny element w sekwencji.
- void display_prev_element ()

Wyświetla poprzedni element w sekwencji.

- void clear_list ()

  Usuwa wszystkie elementy z listy, całkowicie ją czyszcząc i zwalniając pamięć.

- Iterator begin ()

  Zwraca iterator ustawiony na początek listy.

- Iterator end ()

  Zwraca iterator reprezentujący koniec listy.

3.1.1 Detailed Description

Główna klasa zarządzająca listą dwukierunkową.

Klasa przechowuje wskaźniki na początek i koniec listy. Zarządza dodawaniem, usuwaniem i wyświetlaniem elementów, wykorzystując do tego wzorzec Factory oraz wydając iteratory (wzorzec Iterator).

3.1.2 Constructor & Destructor Documentation

3.1.2.1 DoublyLinkedList()

```
DoublyLinkedList::DoublyLinkedList ()
```

Konstruktor klasy.

Konstruktor domyślny klasy DoublyLinkedList.

Inicjalizuje pustą listę, ustawiając wskaźniki na nullptr oraz rozmiar na 0.

3.1.2.2 ~DoublyLinkedList()

```
DoublyLinkedList::~DoublyLinkedList ()
```

Destruktor klasy.

Destruktor domyślny klasy DoublyLinkedList.

3.1.3 Member Function Documentation

3.1.3.1 begin()

```
Iterator DoublyLinkedList::begin ()
```

Zwraca iterator ustawiony na początek listy.

Zwraca iterator wskazujący na początek listy.

Returns

Iterator wskazujący na element head.

Iterator ustawiony na węzeł head.

3.1.3.2 clear_list()

```
void DoublyLinkedList::clear_list ()
```

Usuwa wszystkie elementy z listy, całkowicie ją czyszcząc i zwalniając pamięć.

Czyści listę usuwając wszystkie jej elementy.

3.1.3.3 display_list()

```
void DoublyLinkedList::display_list ()
```

Wyświetla zawartość całej listy od początku do końca.

Wyświetla zawartość listy od początku do końca, korzystając z iteratora.

3.1.3.4 display_next_element()

```
void DoublyLinkedList::display_next_element ()
```

Wyświetla następny element w sekwencji.

Przesuwa kursor na następny element i wyświetla jego wartość.

3.1.3.5 display_prev_element()

```
void DoublyLinkedList::display_prev_element ()
```

Wyświetla poprzedni element w sekwencji.

Przesuwa kursor na poprzedni element i wyświetla jego wartość.

3.1.3.6 display_reversed_list()

```
void DoublyLinkedList::display_reversed_list ()
```

Wyświetla zawartość całej listy w odwrotnej kolejności (od końca do początku).

Wyświetla zawartość listy od końca do początku, korzystając z iteratora.

3.1.3.7 end()

```
Iterator DoublyLinkedList::end ()
```

Zwraca iterator reprezentujący koniec listy.

Zwraca iterator wskazujący na koniec listy (za ostatni element).

Returns

Iterator ustawiony za ostatnim elementem (najczęściej wskazujący na nullptr).

Iterator ustawiony na nullptr.

3.1.3.8 pop_back()

```
void DoublyLinkedList::pop_back ()
```

Usuwa ostatni element z listy i zwalnia jego pamięć.

Usuwa element z końca listy.

3.1.3.9 pop_front()

```
void DoublyLinkedList::pop_front ()
```

Usuwa pierwszy element z listy i zwalnia jego pamięć.

Usuwa element z początku listy.

3.1.3.10 pop_wherever()

```
void DoublyLinkedList::pop_wherever (
            int index)
```

Usuwa element znajdujący się pod wskazanym indeksem.

Usuwa element z podanego indeksu.

Parameters

index	Pozycja elementu do usunięcia (liczona od 0).
index	Pozycja elementu do usunięcia.

3.1.3.11 push_back()

```
void DoublyLinkedList::push_back (
            int value)
```

Dodaje nowy element na sam koniec listy.

Dodaje nowy element na koniec listy.

Parameters

value	Wartość liczbowa do zapisania w nowym węźle.
value	Wartość do dodania.

3.1.3.12 push_front()

```
void DoublyLinkedList::push_front (
            int value)
```

Dodaje nowy element na sam początek listy.

Dodaje nowy element na początek listy.

Parameters

value	Wartość liczbowa do zapisania w nowym węźle.
value	Wartość do dodania.

3.1.3.13 push_wherever()

```
void DoublyLinkedList::push_wherever (
            int index,
            int value)
```

Wstawia nowy element pod wskazany indeks.

Parameters

index	Pozycja, na którą ma zostać wstawiony element (liczona od 0).
value	Wartość liczbowa do zapisania w nowym węźle.
index	Pozycja, na której ma zostać wstawiony element.
value	Wartość do dodania.

The documentation for this class was generated from the following files:

- DoublyLinkedList.h
- DoublyLinkedList.cpp

3.2 Factory Class Reference

Klasa implementująca wzorzec Factory.

```
#include <Factory.h>
```

Public Member Functions

- Node * createNode (int value)

  Tworzy nowy obiekt węzła na stercie.

3.2.1 Detailed Description

Klasa implementująca wzorzec Factory.

Odpowiada za bezpieczne tworzenie nowych węzłów (Node) na stercie. Zastosowanie tego wzorca pozwala na oddzielenie logiki alokacji pamięci od głównej logiki zarządzania samą listą dwukierunkową.

3.2.2 Member Function Documentation

3.2.2.1 createNode()

```
Node * Factory::createNode (
            int value)
```

Tworzy nowy obiekt węzła na stercie.

Alokuje dynamicznie pamięć dla nowego węzła i zwraca na niego wskaźnik.

Wykorzystuje operator new do alokacji pamięci dla nowego węzła i inicjuje go przekazaną wartością.

Parameters

value	Wartość całkowita liczby, która zostanie przypisana do nowego węzła.

Returns

Node∗ Wskaźnik na nowo utworzony węzeł.

The documentation for this class was generated from the following files:

- Factory.h
- Factory.cpp

3.3 Iterator Class Reference

Klasa implementująca wzorzec Iterator dla listy dwukierunkowej.

```
#include <Iterator.h>
```

Public Member Functions

- Iterator (Node ∗current)
  Konstruktor inicjujący iterator.
- void operator++ ()
  Operator inkrementacji.
- void operator-- ()
  Operator dekrementacji.
- int operator∗ ()
  Operator wyłuskania.
- bool operator!= (const Iterator &other)
  Operator nierówności.

3.3.1 Detailed Description

Klasa implementująca wzorzec Iterator dla listy dwukierunkowej.

Pozwala przechodzenie przez elementy listy w obu kierunkach za pomocą standardowych operatorów, ukrywając przed użytkownikiem logikę wskaźników.

3.3.2 Constructor & Destructor Documentation

3.3.2.1 Iterator()

```
Iterator::Iterator (
            Node * current)
```

Konstruktor inicjujący iterator.

Przypisuje wskaźnik na węzeł początkowy do zmiennej current.

Parameters

current	Wskaźnik na węzeł startowy, od którego iterator rozpocznie pracę.

3.3.3 Member Function Documentation

3.3.3.1 operator"!=()

```
bool Iterator::operator!= (
            const Iterator & other)
```

Operator nierówności.

Sprawdza, czy węzły wskazywane przez dwa iteratory są od siebie różne.

Porównuje dwa iteratory. W pętlach, sprawdza czy iterator nie dotarł jeszcze do końca kolekcji.

Parameters

other	Inny obiekt iteratora, z którym porównujemy obecny.

Returns

true Jeśli iteratory wskazują na różne węzły.

false Jeśli iteratory wskazują na ten sam węzeł.

3.3.3.2 operator∗()

```
int Iterator::operator* ()
```

Operator wyłuskania.

Zwraca wartość z węzła wskazywanego przez iterator.

Returns

Liczba całkowita przechowywana w aktualnie wskazywanym węźle.

3.3.3.3 operator++()

```
void Iterator::operator++ ()
```

Operator inkrementacji.

Przesuwa iterator na kolejny element za pomocą wskaźnika next.

Przesuwa wskaźnik iteratora na następny element listy.

3.3.3.4 operator--()

```
void Iterator::operator-- ()
```

Operator dekrementacji.

Cofa iterator na poprzedni element za pomocą wskaźnika prev.

Przesuwa wskaźnik iteratora na poprzedni element listy.

The documentation for this class was generated from the following files:

- Iterator.h
- Iterator.cpp

3.4 Node Class Reference

Klasa reprezentująca pojedynczy węzeł listy dwukierunkowej.

```
#include <Node.h>
```

Public Member Functions

- Node (int value)

  Konstruktor inicjujący nowy węzeł.

Public Attributes

- int **value**

  Przechowywana wartość całkowitą.
- Node ∗ **next**

  Wskaźnik na następny element na liście.
- Node ∗ **prev**

  Wskaźnik na poprzedni element na liście.

3.4.1 Detailed Description

Klasa reprezentująca pojedynczy węzeł listy dwukierunkowej.

Służy jako podstawowy kontener przechowujący dane (liczbę całkowitą) oraz informacje o sąsiadujących elementach w strukturze listy.

3.4.2 Constructor & Destructor Documentation

3.4.2.1 Node()

```
Node::Node (
            int value)
```

Konstruktor inicjujący nowy węzeł.

Tworzy nowy węzeł z zadaną wartością.

Parameters

value	Wartość początkowa przypisywana do zmiennej value.

Inicjuje węzeł ustawiając jego wartość na tę przekazaną w parametrze. Wskaźniki next i prev są bezpiecznie ustawiane na nullptr, ponieważ świeżo utworzony węzeł nie jest jeszcze połączony z resztą listy.

Parameters

value	Wartość liczbowa do przechowania w węźle.

The documentation for this class was generated from the following files:

- Node.h
- Node.cpp

Chapter 4

File Documentation

4.1 DoublyLinkedList.cpp File Reference

Implementacja metod klasy DoublyLinkedList.

```
#include <iostream>
#include "DoublyLinkedList.h"
```

4.1.1 Detailed Description

Implementacja metod klasy DoublyLinkedList.

4.2 DoublyLinkedList.h

```
00001 #ifndef LISTADWUKIERUNKOWA_PYRDOL_JAKUB_DOUBLYLINKEDLIST_H
00002 #define LISTADWUKIERUNKOWA_PYRDOL_JAKUB_DOUBLYLINKEDLIST_H
00003 #include "Factory.h"
00004 #include "Iterator.h"
00005 #include "Node.h"
00006
00015 class DoublyLinkedList
00016 {
00017 private:
00021     Node* head;
00022
00026     Node* tail;
00027
00031     Factory factory;
00032
00036     int size;
00037
00041     Node* cursor;
00042
00043 public:
00047     DoublyLinkedList();
00051     ~DoublyLinkedList();
00055     DoublyLinkedList(const DoublyLinkedList&) = delete;
00056     DoublyLinkedList& operator=(const DoublyLinkedList&) = delete;
00062     void push_front(int value);
00063
00069     void push_back(int value);
00070
00077     void push_wherever(int index, int value);
00078
00082     void pop_front();
00083
```

```
00087     void pop_back();
00088
00094     void pop_wherever(int index);
00095
00099     void display_list();
00100
00104     void display_reversed_list();
00105
00109     void display_next_element();
00110
00114     void display_prev_element();
00115
00119     void clear_list();
00120
00126     Iterator begin();
00127
00133     Iterator end();
00134 };
00135
00136 #endif //LISTADWUKIERUNKOWA_PYRDOL_JAKUB_DOUBLYLINKEDLIST_H
```

4.3 Factory.cpp File Reference

Implementacja metod klasy Factory.

```
#include "Factory.h"
```

4.3.1 Detailed Description

Implementacja metod klasy Factory.

4.4 Factory.h

```
00001 #ifndef LISTADWUKIERUNKOWA_PYRDOL_JAKUB_FACTORY_H
00002 #define LISTADWUKIERUNKOWA_PYRDOL_JAKUB_FACTORY_H
00003 #include "Node.h"
00004
00012 class Factory
00013 {
00014 public:
00024     Node* createNode(int value);
00025 };
00026
00027 #endif //LISTADWUKIERUNKOWA_PYRDOL_JAKUB_FACTORY_H
```

4.5 Iterator.cpp File Reference

Implementacja metod klasy Iterator.

```
#include "Iterator.h"
```

4.5.1 Detailed Description

Implementacja metod klasy Iterator.

4.6 Iterator.h

```
00001 #ifndef LISTADWUKIERUNKOWA_PYRDOL_JAKUB_ITERATOR_H
00002 #define LISTADWUKIERUNKOWA_PYRDOL_JAKUB_ITERATOR_H
00003 #include "Node.h"
00004
00011 class Iterator
00012 {
00013 private:
00017     Node* current;
00018
00019 public:
00025     Iterator(Node* current);
00026
00032     void operator++();
00033
00039     void operator--();
00040
00046     int operator*();
00047
00058     bool operator!=(const Iterator& other);
00059 };
00060
00061 #endif //LISTADWUKIERUNKOWA_PYRDOL_JAKUB_ITERATOR_H
```

4.7 main.cpp File Reference

Plik główny programu testujący implementację dwukierunkowej listy powiązanej.

```
#include <iostream>
#include "DoublyLinkedList.h"
```

Functions

- int main ()

  Główna funkcja programu.

4.7.1 Detailed Description

Plik główny programu testujący implementację dwukierunkowej listy powiązanej.

4.7.2 Function Documentation

4.7.2.1 main()

```
int main ()
```

Główna funkcja programu.

Przeprowadza serię testów metod klasy DoublyLinkedList, w tym dodawanie, usuwanie, wyświetlanie elementów oraz użycie kursora.

Returns

Wartość 0 w przypadku pomyślnego zakończenia programu.

4.8 Node.cpp File Reference

Plik zawierający implementację metod klasy Node.

```
#include "Node.h"
```

4.8.1 Detailed Description

Plik zawierający implementację metod klasy Node.

4.9 Node.h

```
00001 #ifndef LISTADWUKIERUNKOWA_PYRDOL_JAKUB_NODE_H
00002 #define LISTADWUKIERUNKOWA_PYRDOL_JAKUB_NODE_H
00003
00010 class Node
00011 {
00012 public:
00016     int value;
00017
00021     Node* next;
00022
00026     Node* prev;
00027
00033     Node(int value);
00034 };
00035
00036 #endif
```

Index

~DoublyLinkedList
    DoublyLinkedList, 6

begin
    DoublyLinkedList, 6

clear_list
    DoublyLinkedList, 6
createNode
    Factory, 10

display_list
    DoublyLinkedList, 7
display_next_element
    DoublyLinkedList, 7
display_prev_element
    DoublyLinkedList, 7
display_reversed_list
    DoublyLinkedList, 7
DoublyLinkedList, 5
    ~DoublyLinkedList, 6
    begin, 6
    clear_list, 6
    display_list, 7
    display_next_element, 7
    display_prev_element, 7
    display_reversed_list, 7
    DoublyLinkedList, 6
    end, 7
    pop_back, 7
    pop_front, 8
    pop_wherever, 8
    push_back, 8
    push_front, 8
    push_wherever, 9
DoublyLinkedList.cpp, 15

end
    DoublyLinkedList, 7

Factory, 9
    createNode, 10
Factory.cpp, 16

Iterator, 10
    Iterator, 11
    operator!=, 11
    operator++, 12
    operator--, 12
    operator*, 11
Iterator.cpp, 16

main
    main.cpp, 17
main.cpp, 17
    main, 17

Node, 12
    Node, 13
Node.cpp, 18

operator!=
    Iterator, 11
operator++
    Iterator, 12
operator--
    Iterator, 12
operator*
    Iterator, 11

pop_back
    DoublyLinkedList, 7
pop_front
    DoublyLinkedList, 8
pop_wherever
    DoublyLinkedList, 8
push_back
    DoublyLinkedList, 8
push_front
    DoublyLinkedList, 8
push_wherever
    DoublyLinkedList, 9